

CSCE 763: Digital Image Processing Homework #3

Instructor: Spring 2026
University of South Carolina
Solutions by: JC Vaught

Due: 1:15 pm EST, Tuesday, Feb 17

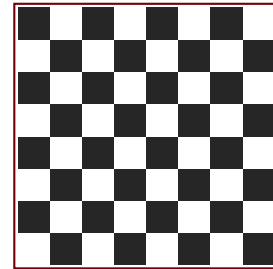
Table of Contents	
Problem 1: Histogram and Blurring (20 pts)	2
Problem 2: Spatial Filtering (20 pts)	6
Problem 3: Averaging Mask and Bar Merging (20 pts)	9
Problem 4: Averaging and Laplacian Order (20 pts)	12
Problem 5: Convolution Filtering (20 pts)	14

Problem 1: Histogram and Blurring (20 pts)

Two $N \times N$ images shown below are quite different, but their histograms are the same. Suppose that each image is blurred with a 3×3 averaging mask. Please ignore the low quality of the images. You can assume all the black regions have a uniform intensity of 0, and the white regions have a uniform intensity of 255.



(a) Half-and-half



(b) Chessboard

- Would the histogram of the blurred images still be equal? Explain.
- If your answer is no, sketch the two histograms.
- Bonus question:** Discuss how the histogram of the blurred image (b) is affected by different image size N . (5 pts)

Step 1: Analyze the Original Histograms

Both images have identical histograms before blurring. Each $N \times N$ image contains exactly $N^2/2$ pixels at intensity 0 (black) and $N^2/2$ pixels at intensity 255 (white). The histogram consists of two impulses at $r = 0$ and $r = 255$, each with height $N^2/2$. **Assumption:** The chessboard in image (b) is a *1-pixel checkerboard*, where each individual pixel alternates between black and white (i.e., pixel (i, j) has intensity 255 if $i + j$ is even, and 0 if $i + j$ is odd). If the checkerboard squares were larger blocks (e.g., $K \times K$ pixels each), then interior pixels of each block would be unaffected by 3×3 averaging (just like the interior of each half in image (a)), and the two histograms would remain similar after blurring.

Step 2: Analyze Blurring of Image (a) — Half-and-Half

The half-and-half image has a single vertical boundary between the white (left) and black (right) halves. After applying a 3×3 averaging mask:

- **Interior white pixels** (more than 1 pixel from the boundary): all 9 neighbors are white. Average = 255. Unchanged.
- **Interior black pixels** (more than 1 pixel from the boundary): all 9 neighbors are black. Average = 0. Unchanged.
- **Boundary pixels** (within 1 pixel of the vertical edge): the 3×3 window straddles white and black regions, producing intermediate values.

Only $3 \times N$ pixels (3 columns near the boundary) are affected. The vast majority of pixels remain at 0 or 255. The histogram has two large spikes at 0 and 255, with a small number of pixels at intermediate values along the boundary.

Step 3: Analyze Blurring of Image (b) — Chessboard

In the chessboard pattern, every pixel is adjacent to pixels of the opposite intensity. For an **interior pixel** (not on the image border), the 3×3 neighborhood in a checkerboard contains:

- If center is white (255): 4 corner neighbors are white, 4 edge neighbors are black $\Rightarrow$ 5 white, 4 black.

$$\text{Average} = \frac{5 \times 255 + 4 \times 0}{9} = \frac{1275}{9} \approx 141.7$$

- If center is black (0): 4 corner neighbors are black, 4 edge neighbors are white $\Rightarrow$ 4 white, 5 black.

$$\text{Average} = \frac{4 \times 255 + 5 \times 0}{9} = \frac{1020}{9} \approx 113.3$$

Every interior pixel is affected, not just boundary pixels. The histogram collapses from two spikes at 0 and 255 to two spikes near 142 and 113.

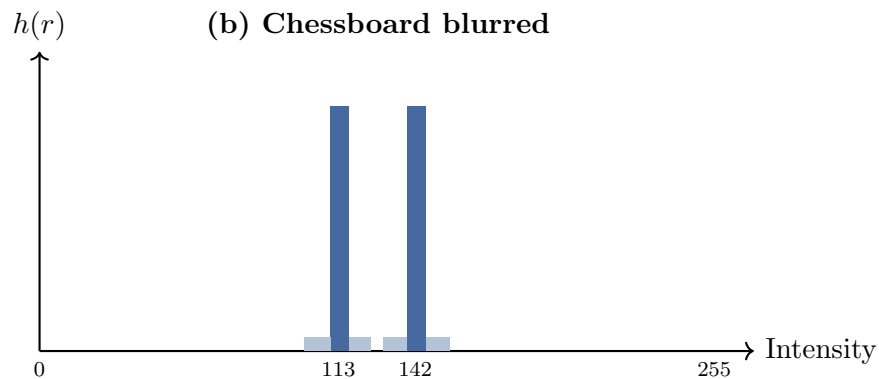
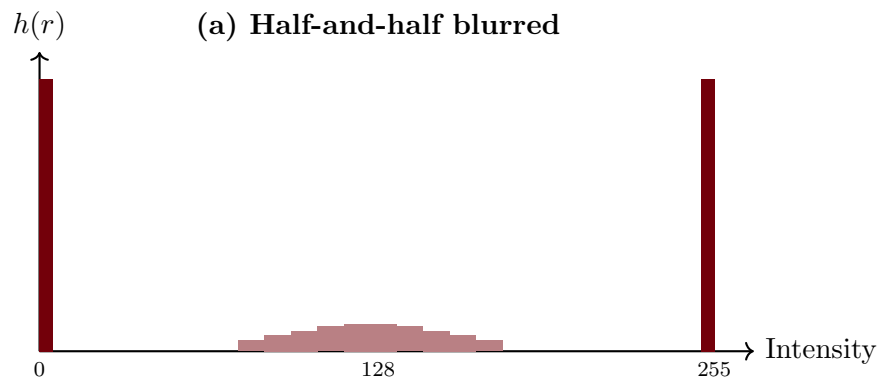
Part (a): Are the Histograms Still Equal?

No. After blurring:

- **Image (a)** retains most pixels at 0 and 255, with only a thin strip of intermediate values at the boundary.
- **Image (b)** has *no* pixels at 0 or 255 (for interior pixels). The entire image shifts to two narrow clusters near 113 and 142.

The histograms are completely different because the 3×3 averaging mask interacts with the *spatial structure* of the image, not just its intensity distribution.

Part (b): Sketch of the Two Blurred Histograms



Part (c) Bonus: Effect of Image Size N on Chessboard Histogram

The histogram of the blurred chessboard depends on the ratio of interior to border pixels.

- **Interior pixels** ($N - 2$ rows $\times$ $N - 2$ columns) produce values ≈ 142 or ≈ 113 as computed above.
- **Border pixels** (on the edges/corners of the image) have fewer than 9 neighbors within the image. With zero-padding, border pixels see additional zeros, shifting their averages toward lower values.

As N increases:

- The fraction of border pixels is $\sim 4N/N^2 = 4/N$, which decreases.
- The histogram converges to two sharp spikes at $\lfloor 1275/9 \rfloor = 142$ and $\lfloor 1020/9 \rfloor = 113$, each containing approximately $N^2/2$ pixels.

For small N , border effects dominate and the histogram is more spread out. In the extreme case $N = 2$, every pixel is a corner pixel and the histogram differs substantially from the two-spike pattern.

Results

(a) No, the histograms of the blurred images are not equal. Image (a) retains large spikes at 0 and 255 with few intermediate values, while image (b) collapses entirely to two narrow peaks near 113 and 142.

(b) See histogram sketches above. Image (a) has a bimodal histogram concentrated at the extremes; image (b) has a bimodal histogram concentrated near the center.

(c) **Bonus:** As $N \rightarrow \infty$, the border-pixel fraction $\rightarrow 0$ and the chessboard histogram converges to two delta functions at ≈ 113 and ≈ 142 . For small N , border effects widen and shift the peaks.

Problem 2: Spatial Filtering (20 pts)

Consider spatial filtering.

- (a) Suppose that you filter an image $f(x, y)$, with a spatial filter mask $w(x, y)$, using convolution, as defined in Eq. (3.4-2), where the mask is smaller than the image in both spatial directions. Show the important property that, if the coefficients of the mask sum to zero, then the sum of all the elements in the resulting convolution array (filtered image) will be zero also (you may ignore computational inaccuracies). Also, you may assume that the border of the image has been padded with the appropriate number of zeros. (10 pts)

$$w(x, y) \otimes f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t) \quad (3.4-2)$$

- (b) Would the result to (a) be the same if the filtering is implemented using correlation, as defined in Eq. (3.4-1)? (10 pts)

$$w(x, y) \odot f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t) \quad (3.4-1)$$

Part (a): Convolution with Zero-Sum Mask

Let $g(x, y) = w(x, y) \otimes f(x, y)$ denote the filtered image. We want to show that if $\sum_{s=-a}^a \sum_{t=-b}^b w(s, t) = 0$, then $\sum_x \sum_y g(x, y) = 0$.

Step 1: Write the Sum of All Output Pixels

$$\sum_x \sum_y g(x, y) = \sum_x \sum_y \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t)$$

Step 2: Exchange the Order of Summation

Since all sums are finite, we can exchange the order:

$$\sum_x \sum_y g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) \left[\sum_x \sum_y f(x - s, y - t) \right]$$

Step 3: Evaluate the Inner Sum

Consider the inner sum $\sum_x \sum_y f(x - s, y - t)$. Substituting $u = x - s$, $v = y - t$ (a change of summation index), and noting that f is zero-padded (i.e., $f(u, v) = 0$ outside the image), the summation range covers all positions where f is nonzero regardless of the shift (s, t) . Therefore:

$$\sum_x \sum_y f(x - s, y - t) = \sum_u \sum_v f(u, v) = S_f \quad (\text{constant, independent of } s, t)$$

where S_f is the sum of all pixel values in the original image.

Step 4: Factor and Conclude

Substituting back:

$$\sum_x \sum_y g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) \cdot S_f = S_f \cdot \underbrace{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t)}_{= 0 \text{ (given)}} = 0 \quad \blacksquare$$

Part (b): Correlation with Zero-Sum Mask

For correlation, $g'(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$. The same argument applies:

$$\sum_x \sum_y g'(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) \left[\sum_x \sum_y f(x + s, y + t) \right]$$

With the substitution $u = x + s$, $v = y + t$, the inner sum is again S_f (independent of s, t):

$$\sum_x \sum_y g'(x, y) = S_f \cdot \sum_{s, t} w(s, t) = S_f \cdot 0 = 0 \quad \blacksquare$$

Yes, the result is the same for correlation. The sign change in the index ($x - s$ vs. $x + s$) does not affect the argument because the inner sum over a shifted version of f always equals S_f under zero-padding.

Results

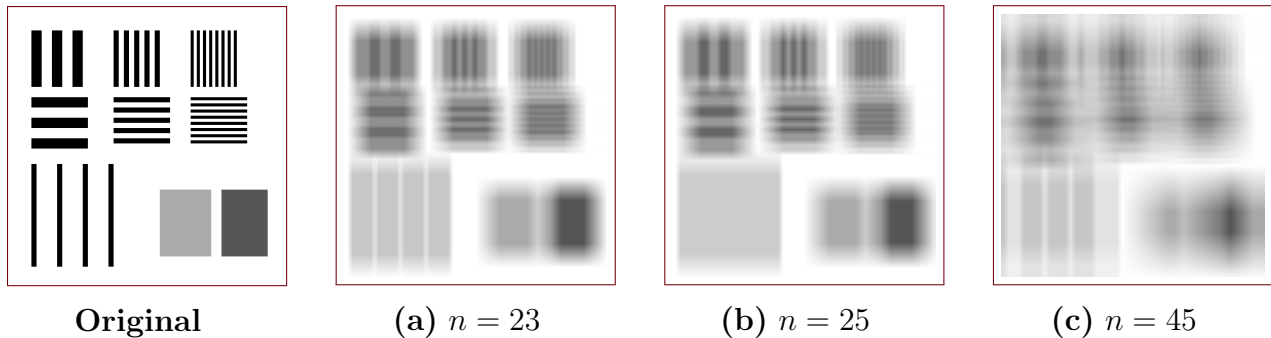
(a) If the mask coefficients sum to zero, i.e., $\sum_{s,t} w(s,t) = 0$, then the sum of all elements in the convolution output is:

$$\sum_{x,y} g(x,y) = S_f \cdot \sum_{s,t} w(s,t) = 0 \quad \blacksquare$$

(b) Yes, the result is identical for correlation. The proof is the same because the inner summation $\sum_{x,y} f(x+s, y+t) = S_f$ is independent of the shift direction. $\blacksquare$

Problem 3: Averaging Mask and Bar Merging (20 pts)

The three images shown were blurred using square averaging masks of sizes $n = 23$, 25, and 45, respectively. The vertical bars on the left lower part of (a) and (c) are blurred, but a clear separation exists between them. However, the bars have merged in image (b), in spite of the fact that the mask that produced this image is significantly smaller than the mask that produced image (c).



Explain the reason for this. The vertical bars are 5 pixels wide, 100 pixels high, and their separation is 20 pixels. (Hints: you can simplify the problem by considering a single scan line through the bars in the image.)

Step 1: Simplify to a 1D Problem

Following the hint, consider a single horizontal scan line through the bars. The 1D intensity profile is a periodic signal:

- Bar width: 5 pixels (intensity 255)
- Gap width: 20 pixels (intensity 0)
- **Period:** $T = 5 + 20 = 25$ pixels

The $n \times n$ averaging mask, applied along this scan line, reduces to a 1D moving average of width n .

Step 2: Effect of a Moving Average on a Periodic Signal

A 1D moving average of width n computes:

$$g(x) = \frac{1}{n} \sum_{k=0}^{n-1} f(x - k)$$

This sums n consecutive pixels. For a periodic signal with period T , the key observation is: if n is an exact multiple of T , then the window always contains the same number of “on” (bar) and “off” (gap) pixels regardless of position, making the output **constant**.

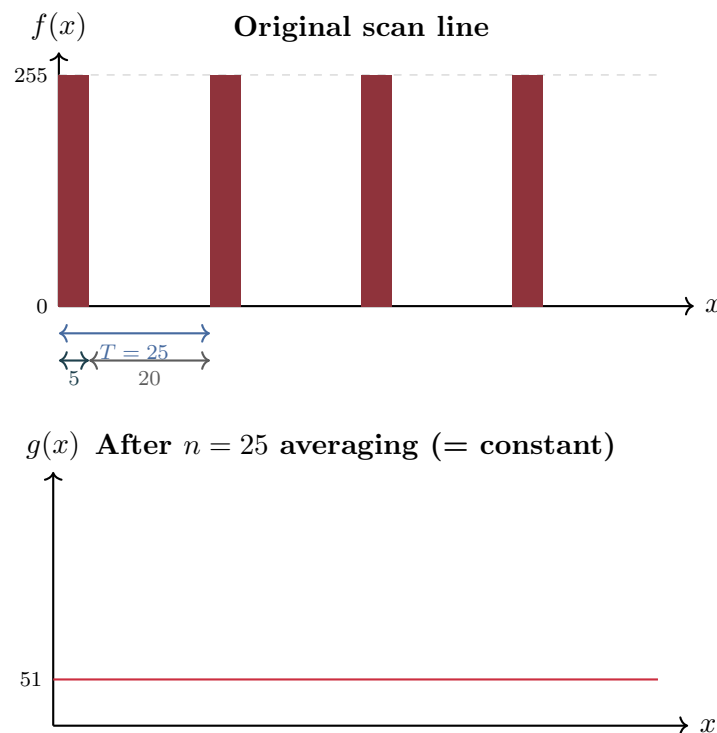
Step 3: Why $n = 25$ Causes Complete Merging

Since $T = 25$ and $n = 25 = 1 \times T$, the averaging window spans **exactly one full period**. At every position, the window contains exactly 5 bar pixels (at 255) and 20 gap pixels (at 0):

$$g(x) = \frac{5 \times 255 + 20 \times 0}{25} = \frac{1275}{25} = 51 \quad (\text{constant})$$

The output is a uniform gray value of 51 along the entire scan line. All spatial variation is destroyed, and the bars **merge completely**.

Visualization: 1D Scan Line Analysis



Step 4: Why $n = 23$ and $n = 45$ Preserve Separation

$n = 23 < T = 25$: The window is smaller than one period. As it slides, the number of bar pixels it covers varies between positions. When centered over a bar, it contains more bar pixels; when centered over a gap, it contains fewer. This variation preserves the intensity modulation, keeping the bars distinguishable.

$n = 45$: Here $45 = 1 \times 25 + 20$. Since 45 is *not* a multiple of the period $T = 25$, the window does not always cover the same number of bar pixels. As the window slides, it alternates between covering 1 and 2 complete bars, producing a varying output. The bars appear more blurred than with $n = 23$ (due to greater smoothing), but the spatial variation is not completely eliminated, so separation is still visible.

Results

The bars merge completely when $n = 25$ because the averaging mask width **exactly matches the period** of the bar pattern ($T = 5 + 20 = 25$ pixels). When $n = T$, the moving average window always contains the same number of bar and gap pixels at every position, producing a constant output and destroying all spatial variation. For $n = 23 < T$ and $n = 45 \neq kT$, the window size does not match the period, so the number of bar pixels in the window varies with position, preserving the intensity modulation and keeping the bars distinguishable. ■

Problem 4: Averaging and Laplacian Order (20 pts)

In a given application an averaging mask is applied to input images to reduce noise, and then a Laplacian mask is applied to enhance small details. Would the result be the same if the order of these operations were reversed?

Step 1: Express Both Operations as Convolutions

Let w_{avg} denote the averaging mask and w_{lap} denote the Laplacian mask. Both are linear spatial filters, so both operations are convolutions:

$$g_1 = w_{\text{lap}} \otimes (w_{\text{avg}} \otimes f) \quad (\text{average first, then Laplacian})$$

$$g_2 = w_{\text{avg}} \otimes (w_{\text{lap}} \otimes f) \quad (\text{Laplacian first, then average})$$

Step 2: Apply Associativity and Commutativity of Convolution

Convolution is both **associative** and **commutative**:

$$w_1 \otimes (w_2 \otimes f) = (w_1 \otimes w_2) \otimes f = (w_2 \otimes w_1) \otimes f = w_2 \otimes (w_1 \otimes f)$$

Applying this to our operations:

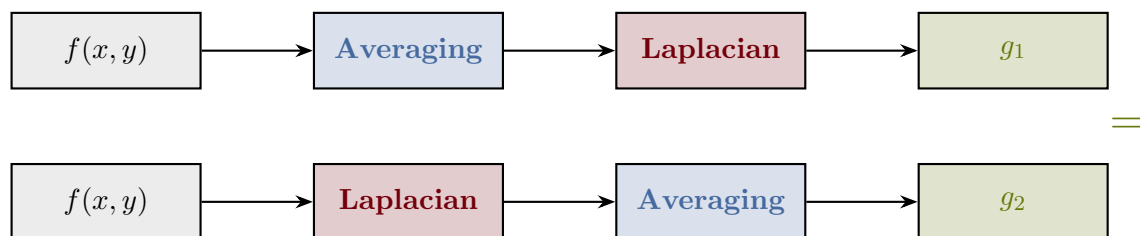
$$g_1 = w_{\text{lap}} \otimes (w_{\text{avg}} \otimes f) = (w_{\text{lap}} \otimes w_{\text{avg}}) \otimes f$$

$$g_2 = w_{\text{avg}} \otimes (w_{\text{lap}} \otimes f) = (w_{\text{avg}} \otimes w_{\text{lap}}) \otimes f$$

Since $w_{\text{lap}} \otimes w_{\text{avg}} = w_{\text{avg}} \otimes w_{\text{lap}}$ (commutativity), we have:

$$g_1 = g_2$$

Visualization: Commutativity of Linear Filters



Results

Yes, the result is the same regardless of the order. Both the averaging mask and the Laplacian mask are linear, shift-invariant operators (i.e., convolutions). Since convolution is associative and commutative:

$$w_{\text{lap}} \otimes (w_{\text{avg}} \otimes f) = w_{\text{avg}} \otimes (w_{\text{lap}} \otimes f)$$

The mathematical output is identical. ■

Problem 5: Convolution Filtering (20 pts)

Given the following 3×3 filter, perform the image filtering using **convolution**. Give the **FULL** filtering result for each filtering process. (Hint: you need to assume an appropriate zero mapping.)

$$w = \begin{bmatrix} -1 & 0 & -1 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

(a) a 2×2 image as shown in image (a) (10 pts)

(b) a 3×3 image as shown in image (b) (10 pts)

1	2
3	4

(a)

0	0	0
0	1	0
0	0	0

(b)

Setup: Convolution Formula

The convolution is defined as:

$$g(x, y) = \sum_{s=-1}^1 \sum_{t=-1}^1 w(s, t) f(x - s, y - t)$$

where $f(x, y) = 0$ outside the image (zero-padding). For a $M \times N$ image convolved with a 3×3 mask, the **full** output size is $(M + 2) \times (N + 2)$.

The mask w with indices $s, t \in \{-1, 0, 1\}$:

$$w(-1, -1) = -1, \quad w(-1, 0) = 0, \quad w(-1, 1) = -1$$

$$w(0, -1) = 0, \quad w(0, 0) = 2, \quad w(0, 1) = 0$$

$$w(1, -1) = 1, \quad w(1, 0) = 0, \quad w(1, 1) = 1$$

Part (a): 2×2 Image — Full 4×4 Output

Image: $f(0,0) = 1$, $f(0,1) = 2$, $f(1,0) = 3$, $f(1,1) = 4$. All other $f = 0$.

Output indices: $x \in \{-1, 0, 1, 2\}$, $y \in \{-1, 0, 1, 2\}$.

Row $x = -1$:

$$g(-1, -1) = w(-1, -1) \cdot f(0, 0) = (-1)(1) = -1$$

$$g(-1, 0) = w(-1, -1) \cdot f(0, 1) = (-1)(2) = -2$$

$$g(-1, 1) = w(-1, 1) \cdot f(0, 0) = (-1)(1) = -1$$

$$g(-1, 2) = w(-1, 1) \cdot f(0, 1) = (-1)(2) = -2$$

Row $x = 0$:

$$g(0, -1) = w(-1, -1) \cdot f(1, 0) = (-1)(3) = -3$$

$$g(0, 0) = w(-1, -1) \cdot f(1, 1) + w(0, 0) \cdot f(0, 0) = (-1)(4) + (2)(1) = -2$$

$$g(0, 1) = w(-1, 1) \cdot f(1, 0) + w(0, 0) \cdot f(0, 1) = (-1)(3) + (2)(2) = 1$$

$$g(0, 2) = w(-1, 1) \cdot f(1, 1) = (-1)(4) = -4$$

Row $x = 1$:

$$g(1, -1) = w(1, -1) \cdot f(0, 0) = (1)(1) = 1$$

$$g(1, 0) = w(1, -1) \cdot f(0, 1) + w(0, 0) \cdot f(1, 0) = (1)(2) + (2)(3) = 8$$

$$g(1, 1) = w(1, 1) \cdot f(0, 0) + w(0, 0) \cdot f(1, 1) = (1)(1) + (2)(4) = 9$$

$$g(1, 2) = w(1, 1) \cdot f(0, 1) = (1)(2) = 2$$

Row $x = 2$:

$$g(2, -1) = w(1, -1) \cdot f(1, 0) = (1)(3) = 3$$

$$g(2, 0) = w(1, -1) \cdot f(1, 1) = (1)(4) = 4$$

$$g(2, 1) = w(1, 1) \cdot f(1, 0) = (1)(3) = 3$$

$$g(2, 2) = w(1, 1) \cdot f(1, 1) = (1)(4) = 4$$

Part (a) Result Grid

-1	-2	-1	-2
-3	-2	1	-4
1	8	9	2
3	4	3	4

Verification: Sum of output = $-1-2-1-2-3-2+1-4+1+8+9+2+3+4+3+4 = 20$.
Sum of mask = 2, sum of image = 10. Product: $2 \times 10 = 20$. ✓

Part (b): 3×3 Image — Full 5×5 Output

Image: $f(1,1) = 1$, all other $f(i,j) = 0$. This is a unit impulse at position $(1,1)$.
Since the only nonzero pixel is $f(1,1) = 1$, each output value $g(x,y)$ has at most one nonzero term in the convolution sum, occurring when $x - s = 1$ and $y - t = 1$, i.e., $s = x - 1$, $t = y - 1$:

$$g(x,y) = w(x-1, y-1) \cdot f(1,1) = w(x-1, y-1)$$

This is nonzero only when $x - 1 \in \{-1, 0, 1\}$ and $y - 1 \in \{-1, 0, 1\}$, i.e., $x \in \{0, 1, 2\}$ and $y \in \{0, 1, 2\}$. All border positions in the 5×5 output are zero.

The central 3×3 block of the output is simply the filter w itself:

Part (b) Result Grid

0	0	0	0	0
0	-1	0	-1	0
0	0	2	0	0
0	1	0	1	0
0	0	0	0	0

Verification: Sum of output = $-1 - 1 + 2 + 1 + 1 = 2$. Sum of mask = 2, sum of image = 1. Product: $2 \times 1 = 2$. ✓

The convolution of a unit impulse with a filter produces the filter itself. This is the fundamental property of the impulse response.

Results

(a) The full 4×4 convolution output for the 2×2 image is:

$$g = \begin{bmatrix} -1 & -2 & -1 & -2 \\ -3 & -2 & 1 & -4 \\ 1 & 8 & 9 & 2 \\ 3 & 4 & 3 & 4 \end{bmatrix}$$

(b) The full 5×5 convolution output for the 3×3 impulse image is:

$$g = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The impulse response equals the filter w itself, embedded in a border of zeros. ■